Predictive Maintenance System

Dataset Exploration with NumPy, Pandas and Matplotlib

This document explains the purpose of the predictive-maintenance data exploration notebook and how the three Python libraries are used to inspect, analyze and visualize machine operating data.

1. System Overview

Predictive maintenance uses equipment data to understand machine condition and investigate possible failure patterns. In this notebook, the system performs exploratory data analysis (EDA) on a CSV dataset. It does not yet train a machine-learning model or make live predictions. Instead, it prepares an understanding of the data that could support a later failure-prediction model.

The workflow is:

1. Load predictive_maintenance.csv into a Pandas DataFrame.
2. Inspect dataset dimensions, columns, and basic data quality.
3. Convert selected numeric measurements to a NumPy array.
4. Calculate descriptive statistics and compare healthy versus failed records.
5. Filter records by torque and tool-wear thresholds for further inspection.
6. Use Matplotlib to display a bar chart comparing healthy and failed machines.

2. Dataset Description

The notebook reports 10,000 records and 10 columns. Each row represents a machine/product observation. The fields include identifiers, product type, machine operating measurements, a binary target label, and a failure-type label.

Column	Meaning	How it is used
UDI	Unique record identifier	Identification; not an operating measurement
Product ID	Product identifier	Identify a product record
Type	Product/machine category label	Categorical information
Air temperature [K]	Air temperature in Kelvin	Operating-condition feature
Process temperature [K]	Process temperature in Kelvin	Operating-condition feature
Rotational speed [rpm]	Rotational speed in revolutions per minute	Operating-condition feature
Torque [Nm]	Torque in newton-metres	Operating-condition feature
Tool wear [min]	Tool wear in minutes	Wear-related feature
Target	0 = no failure, 1 = failure	Binary outcome label
Failure Type	Failure category text	Describes the recorded failure category

Notebook results: 9,661 records have Target = 0 and 339 have Target = 1. The overall recorded failure rate is 3.39%. The notebook's missing-value check reports zero missing values in all 10 columns.

3. How Pandas Was Used

Pandas is used for loading and managing tabular data. The DataFrame keeps the original column names and makes it easy to select columns, inspect records, summarize numeric fields, and filter rows.

Loading the CSV

```
df = pd.read_csv("predictive_maintenance.csv")
```

This reads the CSV file into df, a DataFrame. The file must be in the current working directory or the code must use its correct file path.

Inspecting and summarizing

The notebook uses df.shape to check the number of rows and columns, df.columns.tolist() to list field names, df.describe() to summarize numeric columns, df.isnull().sum() to count missing values, and df['Target'].value_counts() to count each target class.

Filtering records

Pandas is also used to return rows matching a NumPy-generated Boolean mask, for example df.loc[mask]. This lets the analyst inspect records with torque above 50 Nm and tool wear above 150 minutes. The notebook reports 472 matching records; this filter is exploratory and is not itself a confirmed failure rule.

4. How NumPy Was Used

NumPy provides arrays and efficient numerical operations. It is used after loading the data to work directly with numeric measurements and calculate statistics.

Convert the DataFrame to an array: df.to_numpy() creates an array representation of the full table. The notebook reports a shape of (10000, 10), meaning 10,000 rows and 10 columns.

Select numerical features: Five measurements—air temperature, process temperature, rotational speed, torque and tool wear—are converted into a float64 array X with shape (10000, 5).

Calculate descriptive statistics: np.mean, np.median, np.min, np.max and np.std calculate the mean, median, minimum, maximum and standard deviation for each feature. Using axis=0 calculates each statistic column by column.

Count target labels: np.unique(target, return_counts=True) returns the distinct target values and their counts. Since Target is binary (0 or 1), np.mean(target) × 100 gives the percentage of records labelled as failures.

Filter and compare records: Boolean conditions such as (torque > 50) & (wear > 150) create a mask. The notebook also uses target == 0 and target == 1 to split the feature array into healthy and failed groups, then compares their average measurements.

5. How Matplotlib Was Used

Matplotlib is used to create a bar chart of the target distribution. The notebook first counts the values in Target, sorts them by label, and plots two bars labelled Healthy (0) and Failed (1).

```
failure_counts = df["Target"].value_counts().sort_index()
plt.bar(["Healthy (0)", "Failed (1)"], failure_counts.values)
plt.title("Predictive Maintenance: Machine Failure Counts")
plt.xlabel("Machine Status")
plt.ylabel("Number of Machines")
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.show()
```

The chart makes the class imbalance visible: healthy records (9,661) greatly outnumber failure records (339). This matters when planning a future classification model, because accuracy alone could be misleading when failures are relatively rare.

6. Key Findings from the Notebook

- **Dataset size:** 10,000 rows × 10 columns.
- **Missing data:** The notebook reports zero missing values in every column.
- **Target distribution:** 9,661 healthy records and 339 failure records.
- **Failure rate:** 3.39% of records are labelled as failures.
- **High-torque/high-wear subset:** 472 records satisfy torque > 50 Nm and tool wear > 150 minutes.
- **Group comparison:** The notebook's failed group has higher average torque (about 50.17 Nm) and tool wear (about 143.78 minutes) than the healthy group (about 39.63 Nm and 106.69 minutes). These are descriptive differences, not proof of causation.

7. Overall Role of Each Library

Library	Main role	Example operations
Pandas	Load, organize and inspect tabular data	read_csv, head, shape, describe, isnull, value_counts, loc
NumPy	Numerical arrays, calculations and Boolean masks	to_numpy, mean, median, min, max, std, unique
Matplotlib	Visualize the distribution of machine labels	bar, title, xlabel, ylabel, grid, show

8. Conclusion and Next Steps

The notebook demonstrates an initial exploratory data analysis pipeline for predictive maintenance. Pandas handles the CSV and table-oriented inspection, NumPy performs numerical analysis and filtering, and Matplotlib displays the healthy-versus-failed class distribution. This is an analysis and visualization stage, not a complete predictive system.

Possible next steps are to examine failure types, visualize feature distributions, split data into training and test sets, train a classification model, and evaluate it with precision, recall, F1-score and a confusion matrix. Because failures account for only 3.39% of records, evaluation should pay particular attention to recall and false alarms rather than accuracy alone.